

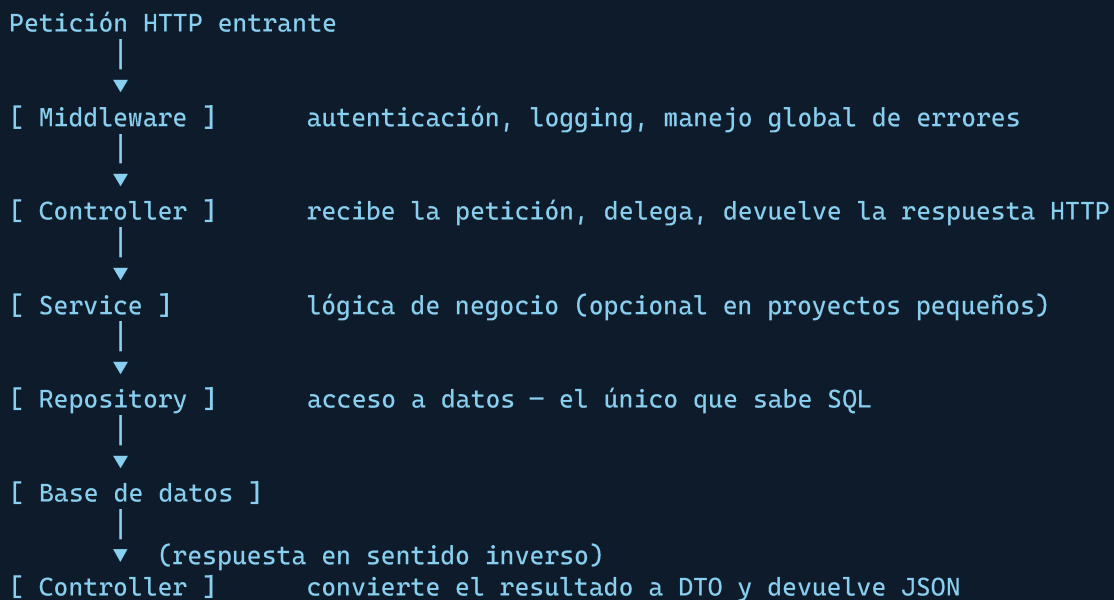
API REST, DTOs y testing con ASP.NET Core

Controladores · Modelos vs DTOs · Arrange/Act/Assert · Postman

En la sesión anterior organizamos los datos y definimos cómo acceder a ellos desde el repositorio. Ahora añadimos la capa que recibe peticiones del exterior: la API REST. También introducimos los tests unitarios como mecanismo para verificar que el código hace exactamente lo que promete.

1. Arquitectura interna de la API

Cada petición HTTP que llega a la aplicación recorre un pipeline de capas antes de generar una respuesta. Entender ese flujo ayuda a saber dónde colocar cada responsabilidad.



Regla de dependencia — cada capa solo conoce a la capa inmediatamente inferior, nunca a las superiores.

El controlador no sabe SQL. El repositorio no sabe nada de HTTP. Eso permite cambiar una capa sin afectar a las otras.

2. El controlador

El controlador es el punto de entrada de cada endpoint. Su responsabilidad es exactamente una: recibir la petición, llamar a quien corresponda y devolver la respuesta HTTP correcta.

2.1 Estructura básica

```

[ApiController]
[Route("api/games")]
public class GamesController : ControllerBase
{
    private readonly IGameRepository _repo;

    public GamesController(IGameRepository repo)
    {
        _repo = repo;
    }

    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        var games = await _repo.GetAllAsync();
        return Ok(games);    // 200
    }

    [HttpGet("{id:int}")]
    public async Task<IActionResult> GetById(int id)
    {
        var game = await _repo.GetByIdAsync(id);
        if (game is null) return NotFound();    // 404
        return Ok(game);    // 200
    }

    [HttpGet("{id:int}/reviews")]
    public async Task<IActionResult> GetReviews(int id)
    {
        if (!await _repo.ExistsAsync(id)) return NotFound();
        var reviews = await _repo.GetReviewsByGameAsync(id);
        return Ok(reviews);    // 200 aunque la lista esté vacía
    }

    [HttpPost("{id:int}/reviews")]
    public async Task<IActionResult> AddReview(int id, [FromBody]
CreateReviewDto dto)
    {
        if (!await _repo.ExistsAsync(id)) return NotFound();
        if (dto.Score < 1 || dto.Score > 10)
            return BadRequest("La nota debe estar entre 1 y 10.");    // 400
        var created = await _repo.AddReviewAsync(id, dto);
        return CreatedAtAction(nameof(GetReviews), new { id }, created);    //
201
    }
}

```

[ApiController]

Este atributo activa comportamientos automáticos que no tendríamos con un controlador MVC normal:

- Binding automático desde el body sin necesidad de [FromBody] explícito en muchos casos.
- Respuesta 400 automática cuando el modelo no pasa las validaciones de DataAnnotations.
- Inferencia de la fuente de binding según la firma del método.

El controlador recibe `IGameRepository` (interfaz), no `GameRepository` (clase concreta).
El contenedor de DI de ASP.NET Core, configurado en `Program.cs`, sabe que cuando alguien pida `IGameRepository` debe crear un `GameRepository`.
En los tests, pasamos directamente un `Mock<IGameRepository>` sin tocar la base de datos.

2.2 Códigos de estado HTTP — referencia práctica

Código	Nombre	Cuándo usarlo en GameShelf
200	OK	GET exitoso. También cuando una lista está vacía — no es un error.
201	Created	POST que crea un recurso nuevo. Se incluye la URL del recurso en el header <code>Location</code> .
400	Bad Request	El cliente envió datos inválidos (score fuera de rango, campo obligatorio vacío).
404	Not Found	El recurso solicitado no existe. El sistema funciona correctamente.
500	Server Error	Excepción no controlada en el servidor. Indica un bug, no una respuesta esperada.

Un 404 no significa que algo haya fallado. Significa que el sistema procesó la petición correctamente y determinó que el recurso no existe. Es una respuesta esperada y documentada.

3. Modelos vs DTOs

Un error habitual en APIs jóvenes es devolver directamente las entidades de la base de datos como respuesta JSON. Funciona en proyectos de juguete, pero crea problemas serios a medida que el proyecto evoluciona.

3.1 El problema concreto en GameShelf

Modelo de base de datos	DTO que recibe el cliente
<pre>public class Game { int Id string Title int ReleaseYear decimal InternalScore // interno string? AdminNotes // sensible ICollection<Review> // referencia circular ICollection<GameGenre> // referencia circular }</pre>	<pre>public class GameDto { int Id string Title int ReleaseYear double? AvgScore // calculado List<string> Genres // aplanado } // Solo lo que el frontend necesita</pre>

Tres razones para usar DTOs:

1. Seguridad — InternalScore y AdminNotes no deben salir de la API nunca. Con el modelo directo, olvidar un campo es fácil.
2. Estabilidad del contrato — si renombramos una columna en la DB, el DTO no cambia. El frontend sigue recibiendo lo mismo.
3. Referencias circulares — Game → Reviews → Game → ... provoca un bucle infinito al serializar a JSON. El DTO aplanar la estructura.

3.2 Los tres DTOs de GameShelf

```
// Respuesta al cliente cuando pide un juego
public class GameDto {
    public int Id { get; set; }
    public string Title { get; set; } = string.Empty;
    public int ReleaseYear { get; set; }
    public double? AvgScore { get; set; } // null si no hay reviews
    public List<string> Genres { get; set; } = new();
}

// Respuesta al cliente cuando pide las reviews de un juego
public class ReviewDto {
    public int Id { get; set; }
    public int Score { get; set; }
    public string? Comment { get; set; }
    public DateTime CreatedAt { get; set; }
}

// Lo que el cliente envía en el body para crear una review
public class CreateReviewDto {
    public int Score { get; set; }
    public string? Comment { get; set; }
}
```

Hay tres DTOs distintos: uno para leer juegos, uno para leer reviews y uno para crear reviews. El que se usa al crear (CreateReviewDto) no tiene id ni fecha — esos campos los asigna el servidor, el cliente no los envía.

4. Introducción al testing en backend

Un test unitario es un fragmento de código que verifica que otro fragmento de código se comporta como se espera, de forma aislada y sin depender de recursos externos (base de datos, red, sistema de archivos).

4.1 El patrón Arrange / Act / Assert

Arrange	Act
Preparamos el escenario. Configuramos los mocks, creamos los objetos necesarios y definimos qué debe devolver cada colaborador.	Ejecutamos la acción. Llamamos al método que queremos probar con los parámetros del escenario que preparamos.

Assert

Comprobamos que el resultado es el esperado. Si algún Assert falla, el test falla y el runner nos indica exactamente qué salió mal.

4.2 Tests del controlador con Moq

En los tests del controlador no queremos tocar la base de datos. Usamos Moq para crear una versión falsa del repositorio que devuelve exactamente lo que necesitamos en cada escenario.

```
// Test 1 - GetById devuelve 404 cuando el juego no existe
[Fact]
public async Task GetById_Returns404_When_Game_Not_Found()
{
    // ARRANGE
    var mockRepo = new Mock<IGameRepository>();
    mockRepo
        .Setup(r => r.GetByIdAsync(999))
        .ReturnsAsync((GameDto?)null);    // simulamos que el juego 999 no
    existe

    var controller = new GamesController(mockRepo.Object);

    // ACT
    var result = await controller.GetById(999);

    // ASSERT
    Assert.IsType<NotFoundResult>(result);
}
```

```
// Test 2 - GetById devuelve 200 con el juego cuando existe
[Fact]
public async Task GetById_Returns200_With_Game_When_Exists()
{
    // ARRANGE
    var mockRepo = new Mock<IGameRepository>();
    mockRepo
        .Setup(r => r.GetByIdAsync(1))
        .ReturnsAsync(new GameDto { Id = 1, Title = "Hollow Knight" });

    var controller = new GamesController(mockRepo.Object);

    // ACT
    var result = await controller.GetById(1);

    // ASSERT
    var ok = Assert.IsType<OkObjectResult>(result);
    var game = Assert.IsType<GameDto>(ok.Value);
    Assert.Equal("Hollow Knight", game.Title);
}
```

Mock<IGameRepository> — crea un objeto que implementa la interfaz pero sin lógica real. Podemos programar exactamente qué devuelve cada método.

Setup + ReturnsAsync — configuramos el comportamiento: "cuando llames a GetByIdAsync con el argumento 1, devuelve este GameDto".

mockRepo.Object — el objeto concreto que pasamos al constructor del controlador. Desde el punto de vista del controlador es indistinguible de un repositorio real.

4.3 Por qué los tests con mock son preferibles a tests con DB

	Tests con mock	Tests con DB real
Velocidad	Milisegundos	Segundos (requiere conexión)
Dependencias	Ninguna	SQL Server, esquema, datos de prueba
Reproducibilidad	Siempre el mismo resultado	Puede variar según el estado de la DB
Qué prueban	La lógica del controlador	La integración completa

5. Pruebas manuales con Postman

Los tests unitarios verifican la lógica interna. Postman nos permite probar la API completa tal como lo haría un cliente real: enviando peticiones HTTP y observando las respuestas.

5.1 Secuencia de pruebas para GameShelf

GET /api/games → 200 OK

Devuelve el array de juegos con avg_score y genres.
Si no hay juegos en la DB, devuelve 200 con [] — no es un 404.

GET /api/games/1 → 200 OK

Devuelve Hollow Knight con sus géneros y nota media. Observa que internal_score y admin_notes no aparecen en la respuesta.

GET /api/games/999 → 404 Not Found

El juego 999 no existe. El cuerpo de la respuesta puede estar vacío o contener un mensaje. El sistema está funcionando correctamente.

GET /api/games/1/reviews → 200 OK

Lista de reviews del juego 1, ordenadas por fecha descendente.

POST /api/games/1/reviews → 201 Created

Body (JSON):

```
{ "score": 9, "comment": "Fantástico" }
```

La respuesta incluye la review creada con el id asignado por la base de datos.

POST /api/games/1/reviews con score = 15 → 400 Bad Request

Body (JSON):

```
{ "score": 15, "comment": "Demasiado bueno" }
```

La validación en el controlador detecta que 15 está fuera del rango 1-10 y rechaza la petición antes de llamar al repositorio.

5.2 Diferencia entre 404 y lista vacía

GET /api/games/999 → 404 (el juego no existe)

GET /api/games/3/reviews → 200 con [] (el juego existe pero no tiene reviews)

Son dos situaciones distintas. En el segundo caso el recurso padre (el juego) existe, simplemente no tiene hijos. Devolver 404 sería incorrecto porque induciría a pensar que el juego 3 no existe.